Prova-T (peso 2,5) – CIC0104 Software Básico – semestre 21/2 – resolvida e comentada

Arquivo original: `SB_Prova_Sistemas.pdf` (3 páginas). É uma prova **já respondida por outro aluno** (nome omitido aqui). As respostas dele **não são gabarito oficial**. Em cada questão eu indico a resposta que considero correta com base nos slides e comparo com a que ele marcou.

Assuntos cobertos

- Arquitetura de von Neumann: componentes, ciclo de busca e execução, gargalo, localidade de referência
- Código absoluto e relocação estática
- Carregadores: absoluto (boot) e de relocação
- Montador: estrutura lógica, passos, entrada e saída
- Ligador: etapas, tipos (linkage editor, linking loader, ligação dinâmica/DLL)
- Sistemas operacionais: tipos (batch, time sharing...), gerência de memória real e virtual (partições, swapping, paginação, segmentação)
- Processos: escalonamento, preempção, deadlock, starvation, região crítica, page fault

Regras da prova (importam para a estratégia)

- Parte 1: 22 itens V/F/NS, 0,32 cada (cerca de 7,0). **Duas erradas anulam uma certa**, **em branco conta como errada**, NS (não sei) vale zero. **Item marcado F precisa de justificativa.**
- Parte 2: 4 discursivas (1,0 + 1,0 + 1,0 + 2,0).
- Estratégia: como um erro custa metade de um acerto (−0,16 contra +0,32), vale a pena chutar quando você tem mais de 1/3 de chance de acertar. Use NS só quando não tiver nenhuma ideia.

Referências usadas (slides)

- `7-teoria-1 de 2.pdf` : von Neumann, código absoluto/relocável, carregador absoluto, montador. A página do PDF é igual ao número do slide.
- `7-teoria-2 de 2.pdf` : ligador, carregador de relocação, DLL. A página do PDF *p* corresponde ao slide impresso *32+p* (ex.: pág. 8 = slide 40).
- `8-so-red-1 de 2.pdf` e `8-so-red-2 de 2.pdf` : sistemas operacionais. Na parte 2, a pág. *p* corresponde ao slide *13+p*.
- `9-boot-so-red.pdf` : boot (pré-boot, ROM BIOS, MBR, NTLDR).

Parte 1 – Questões objetivas

Questão 1

Enunciado: A arquitetura de von Neumann é caracterizada por CPU, memória e periféricos, sendo a CPU composta por unidade lógica e aritmética, unidade de controle, registradores e contador de programas – PC. Na memória RAM são armazenados dados e programas.

Resposta: F (provável). O aluno marcou F.

Comentário:
O slide 5 de `7-teoria-1 de 2.pdf` lista **quatro** elementos na arquitetura de von Neumann:

- CPU (ULA, UC, registradores, PC)
- Memória (dados e programas)
- Periféricos de E/S (podem ser mapeados na memória)
- **Barramentos (dados, endereço e controle)**

O enunciado "caracteriza" a arquitetura e deixa os **barramentos** de fora. A composição da CPU está correta. O slide fala só em "memória", não em "memória RAM", mas isso é detalhe.

Justificativa modelo (F): "A arquitetura de von Neumann é caracterizada por CPU, memória, periféricos de E/S **e barramentos** (dados, endereço e controle)."

Incerteza: o item é igual ao slide com uma única omissão (barramentos). Como esse professor costuma montar os itens copiando o slide e alterando um detalhe, F é a leitura mais provável. Ainda assim, alguém poderia argumentar que uma lista incompleta não é falsa. Compare com a Q14, onde uma omissão parecida **não** torna o item falso.

Dica: decore a lista do slide 5 com os **4** componentes. "CPU, memória e periféricos" sem barramentos é a pegadinha.

Questão 2

Enunciado: A execução é rápida na arquitetura de von Neumann porque dados e instruções são armazenados em blocos de memória adjacentes devido ao fenômeno de localidade de referência.

Resposta: V. O aluno marcou V.

Comentário:
O slide 7 de `7-teoria-1 de 2.pdf` diz quase literalmente: "Execução rápida porque dados e instruções são armazenados em blocos de memória adjacentes → Localidade de referência". Com instruções e dados em blocos contíguos, o programa tende a acessar endereços próximos (localidade espacial) e os mesmos endereços repetidamente (localidade temporal, por exemplo em laços). Isso é o que torna a cache e o pré-carregamento eficientes.

Pegadinha: a frase "devido ao fenômeno de localidade" inverte um pouco a causalidade. A rigor, o armazenamento adjacente **gera** a localidade, e a localidade é o que deixa a execução rápida. Como o texto é praticamente o do slide, a resposta esperada é V.

Questão 3

Enunciado: A função básica do ligador é ligar os módulos e resolver as referências externas, seguindo as seguintes etapas: ler e relocar de forma estática o módulo objeto P; atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; concatenar P aos módulos anteriores; e resolver as referências externas em P.

Resposta: V. O aluno marcou V.

Comentário:

- `7-teoria-2 de 2.pdf` , pág. 2 (slide 34): a "função básica" do ligador é a **ligação de módulos** e a **resolução de referências externas**.
- Pág. 8 (slide 40), "Etapas":
  i. Ler e relocar o módulo objeto P
  ii. Atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos
  iii. Concatenar P aos módulos anteriores
  iv. Resolver as referências externas em P
- "De forma estática" está correto: o ligador faz **relocação estática** (pág. 6, slide 38). Cada módulo é relocado somando a constante k, que é o tamanho dos módulos anteriores (pág. 11, slide 43).

Dica: guarde a ordem das etapas: **ler/relocar → tabela global → concatenar → resolver**.

Questão 4

Enunciado: A redução do gargalo de von Neumann entre CPU e periféricos de I/O pode ser obtida com o uso de memória **cache** e processadores **pipelining** com múltiplos estágios.

Resposta: F. O aluno marcou F, com justificativa correta.

Comentário: o slide 8 de `7-teoria-1 de 2.pdf` separa as técnicas por gargalo:

Gargalo	Técnicas
CPU × memória	memória **cache**; processadores com múltiplos estágios (**pipelining**)
Memória × periféricos de E/S	**blocagem e bufferização**; **DMA**; **canais** de controle de E/S

O item atribui ao gargalo CPU × E/S as técnicas do gargalo CPU × memória.

Justificativa modelo (F): "Cache e pipelining reduzem o gargalo **CPU × memória**. Para memória × E/S usam-se blocagem/bufferização, DMA e canais de E/S."

Pegadinha: essa troca entre os dois gargalos é clássica. Decore a tabela.

Questão 5

Enunciado: Ao se ligar um computador, pressionando o botão de **power on**, um carregador absoluto armazenado em memória volátil é ativado para iniciar o processo de carga do sistema operacional.

Resposta: F. O aluno marcou F, com justificativa correta.

Comentário:

- `7-teoria-1 de 2.pdf`, slide 15: o carregador absoluto "inicia a carga do sistema operacional", é código absoluto e fica "armazenado em memória **não volátil**, em posição fixa e conhecida (ROM, flash)". É ativado pela "ação do operador ao apertar o botão". No mainframe se chama **IPL** (Initial Program Load).
- Faz sentido: se estivesse em memória volátil (RAM), o conteúdo teria se perdido ao desligar.
- `9-boot-so-red.pdf`, slide 6: a CPU executa a ROM BIOS. No endereço FFFF:0000 há um `jmp` para o código de boot.

Justificativa modelo (F): "O carregador absoluto fica em memória **não volátil** (ROM/flash), em endereço fixo."

Questão 6

Enunciado: As otimizações algorítmicas de código que podem ser realizadas na síntese do código são de tempo de execução ou de alocação de memória.

Resposta: F (razoavelmente seguro). O aluno marcou **V**, o que provavelmente está errado.

Comentário:

- `7-teoria-1 de 2.pdf` , slide 31 (Síntese): "Otimiza código (opcional). Otimiza o código intermediário gerado pela análise semântica. **A otimização é do código e não das funções implementadas ao nível do código do programador**."
- Ou seja, a síntese faz otimização **de código** (sobre o código intermediário: eliminar redundâncias, escolher instruções melhores etc.). Ela **não** faz otimização **algorítmica**, que mudaria o algoritmo escrito pelo programador. Trocar um algoritmo $O(n^2)$ por um $O(n \log n)$ é responsabilidade do programador, não do montador/tradutor.
- O item usa termos plausíveis (tempo de execução, memória) para esconder que a palavra errada é "**algorítmicas**".

Justificativa modelo (F): "Na síntese a otimização é **do código intermediário**, não algorítmica. O tradutor não altera os algoritmos/funções implementados pelo programador."

Incerteza: é possível que o professor quisesse V, no sentido de "otimizações de tempo ou de espaço". Mas o slide 31 frisa exatamente que a otimização não é das funções do programador, o que sustenta F.

Questão 7

Enunciado: Carregador de relocação estática lê o tamanho do programa a carregar; solicita ao sistema operacional (S.O.) alocar um bloco contíguo de memória com tamanho suficiente para conter o programa; carrega o programa a partir do endereço dado pelo S.O., reloca os endereços relocáveis; e inicializa o PC com o **entrypoint** do programa.

Resposta: V. O aluno marcou V.

Comentário: `7-teoria-2 de 2.pdf` , pág. 17 (slide 49), carregador de relocação em ambiente monousuário:

1. Lê o tamanho do programa e o entrypoint.
2. Solicita ao gerente de memória um bloco **contíguo** suficiente.
3. Carrega a partir do endereço dado pelo S.O., **relocando os endereços relocáveis** (somando a constante de relocação k, que é o endereço de carga; slide 14 de `7-teoria-1`).
4. Transfere o controle a partir do entrypoint, o que equivale a fazer PC ← entrypoint.

O item reproduz essa sequência.

Dica: a pergunta do slide 49 é "Qual o problema com esse esquema?". Resposta: exige um bloco **contíguo** do tamanho do programa todo, o que causa fragmentação e impede carregar programas maiores que o maior bloco livre. Isso motiva paginação e segmentação.

Questão 8

Enunciado: Código absoluto foi utilizado no início da computação com sistemas monousuário e não é mais utilizado devido a que os sistemas atuais são multiprogramados ou multiprocessados.

Resposta: F. O aluno marcou F, com justificativa correta.

Comentário:

- `7-teoria-1 de 2.pdf` , slide 13: código absoluto usa posições e endereços fixos e reais. Ainda é usado em "**Memória ROM em procedimentos de boot**", por exemplo FFFF:0000 da ROM BIOS.
- O carregador absoluto (slide 15) é código absoluto e roda **toda vez** que o computador é ligado.
- A tabela de `8-so-red-1 de 2.pdf` , pág. 15, também lista "código absoluto" como abordagem de memória em monoprogramação, o que inclui sistemas embarcados atuais.

Justificativa modelo (F): "Código absoluto ainda é usado, por exemplo na ROM BIOS e no carregador absoluto do boot."

Questão 9

Enunciado: Do ponto de vista lógico, em geral um montador é estruturado em um pré-processamento (retira comentários, expande macro e faz a montagem condicional), passo 1 (análises

léxica, sintática e semântica) e passo 2 (otimização e geração de código absoluto).

Resposta: F. O aluno marcou F, mas não justificou.

Comentário:

- **Erro principal:** o montador gera **código objeto relocável**, não código absoluto (`7-teoria-1 de 2.pdf` , slides 17, 18 e 31).
- **Erro secundário:** do ponto de vista **lógico**, a estrutura é **análise** (léxica → sintática → semântica → código intermediário) e **síntese** (otimização → geração de código), conforme o slide 25. A divisão em passos (pré-passo, passo 1, ..., passo final) é a estruturação **física**, ou seja, quantas leituras do fonte são feitas (slide 18).
- O pré-processamento descrito (tira comentários, expande macros, montagem condicional) está correto (slide 26).

Justificativa modelo (F): "A geração é de código objeto **relocável**, não absoluto."

Pegadinha: "código absoluto" no lugar de "relocável" aparece em vários itens. **Montador e ligador produzem código relocável.**

Questão 10

Enunciado: Em um S.O. do tipo **batch** a melhor gerência de memória real é paginação-segmentação.

Resposta: F (provável, com incerteza). O aluno marcou NS.

Comentário:

- Um S.O. batch (processamento em lote, ex.: IBM MVS; `8-so-red-1 de 2.pdf` , pág. 4) roda jobs longos, sem interação e **CPU-intensivos**, com requisitos de memória conhecidos. Para esse perfil, o casamento clássico é **partições** (alocação contígua em multiprogramação; `8-so-red-2` , pág. 4/slide 17). O job inteiro fica carregado e não há o custo de tradução de endereços e de faltas de página.
- Paginação-segmentação aparece nos slides sob o título "Gerência de Memória **Virtual**" (`8-so-red-2` , pág. 10/slide 23). O item fala em memória **real**.
- Tudo indica que o professor trocou as associações: o "par certo" é **batch → partições (CPU-intensivo)** e **time sharing → swapping/paginação**. Compare com a Q11, que usa a mesma frase com o tipo de S.O. trocado.

Justificativa modelo (F): "Batch roda jobs CPU-intensivos e não interativos. A gerência adequada é por **partições**. Paginação-segmentação é técnica de memória virtual."

Incerteza: não há nenhum slide que diga explicitamente "a melhor para batch é X". A resposta depende da visão do professor. Pelo critério de pontuação, com mais de 1/3 de chance vale responder F em vez de NS.

Questão 11

Enunciado: Em um S.O. do tipo time **sharing** como as transações são CPU intensiva, a melhor gerência de memória real é de partições.

Resposta: F. O aluno marcou NS.

Comentário:

- Em time sharing (ex.: IBM TSO, uso de **time slice**; `8-so-red-1` , pág. 4) as transações são **interativas**, curtas e **I/O-intensivas** (o usuário digita, espera resposta). Quem é CPU-intensivo é o **batch**. A premissa do item está errada.
- Para time sharing, os slides associam **swapping**: "troca processos via interrupção (**time slice**)" (`8-so-red-2` , pág. 5/slide 18). Paginação também serve. Partições fixas limitam o número de usuários e fragmentam a memória.

Justificativa modelo (F): "Em time sharing as transações são interativas e I/O-intensivas, não CPU-intensivas. CPU-intensivo é o batch. A gerência adequada é **swapping**/paginação."

Dica: Q10 e Q11 formam um par com as associações trocadas. Batch combina com CPU-intensivo e partições. Time sharing combina com interativo, time slice e swapping/paginação.

Questão 12

Enunciado: Gerência de memória baseada em páginas minimiza a fragmentação interna, favorece a proteção de código e a fragmentação externa.

Resposta: F (razoavelmente seguro). O aluno marcou **V**, o que provavelmente está errado.

Comentário:

- Na paginação a memória é dividida em blocos de **tamanho fixo** (`8-so-red-2` , pág. 7/slide 20). Isso **elimina a fragmentação externa**: qualquer frame livre serve. Por outro lado, gera **fragmentação interna**, porque a última página do processo raramente fica cheia.
- O item inverte as duas coisas: diz que **minimiza a interna** e **favorece a externa**.
- "Favorece a proteção de código" está correto (slide 20: "facilita a proteção contra acesso indevido ao código").
- Quem tem o perfil descrito no item (sem fragmentação interna, com externa, ótima proteção) é a **segmentação**: blocos de tamanho variável, que "facilita em muito a proteção do código" (slide 22).

Justificativa modelo (F): "A paginação **elimina a fragmentação externa** e sofre de **fragmentação interna** (última página incompleta). O comportamento descrito é o da segmentação."

Observação: o slide 13 de `8-so-red-1` diz, para blocos de tamanho fixo, "sem fragmentação", no sentido de sem fragmentação externa. Isso reforça que "favorece a fragmentação externa" é falso.

Questão 13

Enunciado: Ligadores são dos tipos: **Linkage Editor** (gera um arquivo binário que pode ser reusado) ou **Linking Loader/Binder** (carrega o código gerado na memória sem gerar o arquivo que possa ser reusado), ou **carregador de função** de biblioteca de ligação dinâmica (tipo DLL).

Resposta: V (provável). O aluno marcou NS.

Comentário: `7-teoria-2 de 2.pdf` :

- Pág. 16 (slide 48), **Linkage Editor**: liga antes da carga e grava o resultado em arquivo (o **módulo de carga**). É mais eficiente se o programa vai ser executado várias vezes, ou seja, o arquivo é **reusado**.
- Mesmo slide, **Linking Loader ou binder**: após a ligação o programa é carregado na memória e "**não é gerado um arquivo executável**".
- Pág. 18 (slide 50), "Tipos de Ligadores", **Dynamic linking**: a ligação ocorre em tempo de execução, e a rotina é carregada e ligada quando chamada pela primeira vez. No Windows são as **DLLs**. Págs. 19–22: o **carregador** resolve a relocação das funções de biblioteca usando a tabela *fixup*.

Os três tipos batem com os slides.

Incerteza: o nome "carregador de função de biblioteca de ligação dinâmica" não é o termo exato do slide ("Dynamic linking"). Como os slides dizem que é o carregador que resolve as funções da DLL, a descrição é compatível. V é a leitura mais provável.

Questão 14

Enunciado: Na arquitetura de von Neumann o processamento é sequencial e segue o ciclo: busca – decodificação – busca de operandos – execução. O carregador carrega o programa na memória, atribui a PC o valor do **entry point** (endereço de uma instrução) e inicia o ciclo.

Resposta: V (provável). O aluno marcou **F**, justificando: "não diz sobre incrementação do PC".

Comentário:

- `7-teoria-1 de 2.pdf` , slide 7: "Processamento é seqüencial. Ciclo busca – decodificação – busca de operandos – execução (fetch–execute cycle) e PC. **PC** ← **Entry point**. PC é incrementado a cada ciclo de busca."
- Tudo o que o item afirma está correto: processamento sequencial, as 4 fases na ordem certa, PC ← entry point feito pelo carregador (ver também Q7).
- Deixar de citar o incremento do PC é **omissão**, não afirmação errada. Para o item ser falso, teria que dizer algo como "o PC não é incrementado" ou trocar a ordem do ciclo.

Incerteza: depende do critério do professor para omissões. Na Q1 a omissão dos barramentos tende a tornar o item F porque o item se propõe a **caracterizar** a arquitetura (definição incompleta). Aqui o item só descreve o ciclo e a ação do carregador, sem pretender ser completo. Considero V mais provável. A justificativa do aluno ("não diz sobre incrementação") é fraca para um F.

Questão 15

Enunciado: O advento do código relocável permitiu o uso de bibliotecas de código objeto como é o caso do **bytecode** Java que roda em qualquer computador com uma JVM Java instalada.

Resposta: F (razoavelmente seguro). O aluno marcou NS.

Comentário:

- A primeira parte é verdadeira. Os slides 14 e 23 de `7-teoria-1 de 2.pdf` dizem: "Relocação estática **viabiliza a construção de bibliotecas de código objeto**" e "É a existência da

relocação que possibilita o uso de bibliotecas de código objeto".
- O exemplo está errado. **Bytecode Java não é código objeto relocável** (código de máquina de uma arquitetura real). É um código intermediário para uma **máquina virtual** (JVM), que o interpreta ou compila em tempo de execução (JIT). Ele roda "em qualquer computador" graças à **JVM** (portabilidade), não à relocação.

Justificativa modelo (F): "Bytecode Java não é código objeto relocável. É código intermediário executado pela máquina virtual Java. Sua portabilidade vem da JVM, não da relocação."

Pegadinha: a primeira oração é verdadeira e "contamina" o item. Leia até o fim: o exemplo final é que o torna falso.

Questão 16

Enunciado: Para maximizar o **throughput** e evitar **deadlock** um processo só pode entrar na fila de prontos se já estiver com todos os recursos que necessitar.

Resposta: F. O aluno marcou F, com justificativa correta.

Comentário:

- Exigir todos os recursos antes de executar é uma técnica de **prevenção de deadlock**: nega a condição de "posse e espera" (hold and wait). Isso **evita deadlock**.
- Mas **não maximiza o throughput**, pelo contrário:
  - os recursos ficam alocados e ociosos por muito tempo (baixa utilização);
  - processos esperam muito para entrar na fila de prontos, o que reduz o paralelismo e o throughput;
  - pode causar **starvation** de processos que precisam de muitos recursos.
- Slides relacionados: `8-so-red-1` , pág. 9 (o objetivo é maximizar o throughput) e pág. 10 ("mecanismos podem ser implementados com o objetivo de evitar situações de deadlock").

Justificativa modelo (F): "A técnica evita deadlock, mas **reduz o throughput**: recursos ficam alocados e ociosos, e processos esperam muito para executar."

Questão 17

Enunciado: Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos.

Resposta: V (provável na visão do professor, mas **incerto**). O aluno marcou V, porém colocou uma justificativa de F para este item, o que é **contraditório**.

Comentário:

- `8-so-red-1 de 2.pdf` , pág. 10 (slide 10): o S.O. controla a alocação de um recurso. Se o recurso está alocado, o processo é bloqueado. Se está disponível, é alocado e o controle volta ao processo. O termo "**Região crítica**" aparece como subitem desse mecanismo. Nos slides, a região crítica aparece associada ao mecanismo que garante que só um processo use o recurso, ou seja, à exclusão mútua.
- Há também o construto de linguagem "**região crítica**" (Brinch Hansen/Hoare: `region v do S`). É um mecanismo de sincronização que **garante** a exclusão mútua, o que sustenta o V.
- **Contra-argumento (por que pode ser F):** na definição mais comum dos livros de SO, região crítica é o trecho que **acessa o recurso compartilhado** e, por isso, **precisa** ser executado em exclusão mútua. Quem garante a exclusão são os protocolos de entrada e saída (semáforos, mutex, monitores), não a região em si.
- **Sobre a justificativa do aluno:** "região crítica é ... onde pode ocorrer um deadlock" está **errado**. Sem exclusão mútua, o que acontece na região crítica é **condição de corrida** (race condition). Deadlock é espera circular por recursos. E se ele marcou V, não deveria justificar.

Dica: se marcar F, justifique: "Região crítica é o trecho que **acessa recursos compartilhados** e que deve ser executado em exclusão mútua. Quem garante a exclusão são os mecanismos de sincronização (semáforo, mutex, monitor)."

Questão 18

Enunciado: Se um processo de posse da CPU solicitar acesso a uma página não disponível na memória será criado um programa de canal para carregar a página em uma **page frame** e o processo ficará bloqueado.

Resposta: V. O aluno marcou V.

Comentário: isso é uma **falta de página** (page fault), tratada como uma operação de E/S.

- `7-teoria-1 de 2.pdf` , slide 9 ("Resolução de uma operação de E/S", por interrupção):
  i. A informação não está disponível, então o processo é **bloqueado** numa fila específica.
  ii. Gera-se chamada ao Supervisor, que **cria um programa para o canal de E/S**.
  iii. O canal gera interrupção ao terminar, e o processo é desbloqueado e volta para a fila de **prontos**.

- `8-so-red-1` , pág. 11: "As operações a serem executadas em um canal de E/S são definidas em um **programa para o canal**".
- `8-so-red-2` , pág. 7 (slide 20): na paginação, só a página com o entrypoint é carregada inicialmente. As demais vêm sob demanda, e o LRU escolhe a página a remover quando não há frame livre.

Questão 19

Enunciado: Sistemas de escalonamento procuram maximizar o **throughput** e minimizar o **response time** e o **turnaround time**. Por isso um bom escalonador deve ser não preemptivo.

Resposta: F. O aluno marcou F.

Comentário:

- A primeira frase é literal do slide (`8-so-red-1` , pág. 9: "Sistemas de escalonamento procuram maximizar o throughput e minimizar o turnaround time e o response time").
- O mesmo slide diz que o "escalonamento pode ser **preemptivo ou não preemptivo**". Não existe a regra "deve ser não preemptivo".
- Para minimizar o **response time** (sistemas interativos), a **preempção** (time slice, round robin) é justamente o que ajuda. Num escalonador não preemptivo, um processo longo segura a CPU e todos os outros esperam.
- A conclusão "por isso" não decorre da premissa.

Justificativa modelo (F): "O escalonamento pode ser preemptivo ou não. Para minimizar o response time a preempção (time slice) é vantajosa. A conclusão não decorre da premissa."

Sobre a justificativa do aluno: "um bom escalonador deve ser preemptivo" é absoluto demais. Batch, por exemplo, pode usar não preemptivo. Prefira "pode ser preemptivo ou não preemptivo".

Questão 20

Enunciado: Um escalonamento de processos adequado para um S.O. multiprocessado é o **round-robin** pois permite evitar **deadlock.**

Resposta: F. O aluno marcou F, mas a justificativa está parcialmente errada.

Comentário:

- Round-robin (time slice com fila circular) **evita starvation**, porque todo processo recebe sua fatia de tempo periodicamente. **Não evita deadlock.**
- Deadlock é um problema de **alocação de recursos** (espera circular por recursos), não de escalonamento da CPU. Processos em deadlock ficam **bloqueados** e nem estão na fila de prontos, então o round-robin nem chega a escalá-los.
- Deadlock se trata com prevenção (negar uma das condições: exclusão mútua, posse e espera, não preempção de recursos, espera circular), com prevenção dinâmica (ex.: algoritmo do banqueiro) ou com detecção e recuperação.

Justificativa modelo (F): "Round-robin evita **starvation**, não deadlock. Deadlock depende da política de alocação de recursos."

Sobre a justificativa do aluno: "não garante a justiça de atendimento" está **errado**. O round-robin é justamente o algoritmo **justo** (fair).

Questão 21

Enunciado: Um ligador, a partir de arquivos de módulos objetos relocáveis, resolve as referências externas e gera um arquivo contendo os códigos objetos ligados, e com a indicação dos endereços relocáveis. Esse arquivo, união dos códigos objetos, não é executável.

Resposta: F. O aluno marcou F.

Comentário:

- `7-teoria-2 de 2.pdf` , pág. 2 (slide 34): Saída do ligador: "arquivo binário com código de máquina relocável **executável**" e "endereços relocáveis".
- O arquivo gerado é **executável**. Ele mantém a lista de endereços relocáveis para que o **carregador de relocação** ajuste os endereços na carga (Q7).
- Detalhe: o linking loader não gera arquivo (slide 48), mas aí o item nem se aplica, já que ele diz "gera um arquivo".

Justificativa modelo (F): "O arquivo gerado pelo ligador é **executável** (relocável). O carregador de relocação apenas ajusta os endereços relocáveis na carga."

Questão 22

Enunciado: Um montador é um programa que converte um programa em código Assembly em um código objeto relocável (código de máquina), se o programa Assembly estiver correto, e cria um arquivo binário com código de máquina relocável, contendo uma tabela (ou lista) de endereços relocáveis e uma lista de símbolos não resolvidos pelo montador.

Resposta: V. O aluno marcou V.

Comentário: é praticamente uma cópia de `7-teoria-1 de 2.pdf` , slide 17:

- **Definição:** "programa que converte um programa Assembly em um código objeto relocável (código de máquina) se o programa Assembly estiver correto".
- **Saída:** arquivo binário com código de máquina relocável, tabela (ou lista) de endereços relocáveis e lista de referências externas (símbolos não resolvidos pelo montador).
- Se houver erro de sintaxe, o montador não gera o código e emite mensagem de erro.

Dica: compare montador e ligador:

- **Montador:** Assembly → objeto relocável + lista de relocáveis + lista de referências externas.
- **Ligador:** objetos relocáveis → **executável** relocável + endereços relocáveis (referências externas resolvidas).

Resumo da Parte 1 (minha resposta × aluno)

Q	Resposta sugerida	Aluno	Situação
1	F (provável)	F	ok
2	V	V	ok
3	V	V	ok
4	F	F	ok
5	F	F	ok
6	**F**	V	aluno provavelmente errou
7	V	V	ok
8	F	F	ok

Q	Resposta sugerida	Aluno	Situação
9	F	F	ok (sem justificativa)
10	F (provável)	NS	–
11	F	NS	–
12	**F**	V	aluno provavelmente errou
13	V (provável)	NS	–
14	**V** (provável)	F	discutível
15	F	NS	–
16	F	F	ok
17	V (incerto)	V	ok, mas justificativa contraditória
18	V	V	ok
19	F	F	ok
20	F	F	ok (justificativa parcialmente errada)
21	F	F	ok
22	V	V	ok

Parte 2 – Questões discursivas

Discursiva 1 (1,0)

Enunciado: Caracterize o que é (0,5) e o que causa localidade de referência (0,5).

Resposta modelo:

O que é (0,5): localidade de referência é a tendência de um programa, num curto intervalo de tempo, acessar repetidamente **as mesmas posições de memória** ou **posições próximas** às acessadas recentemente. Tem duas formas:

- **Localidade temporal:** um endereço acessado agora tende a ser acessado de novo em breve (variáveis de laço, contadores, instruções de um loop).
- **Localidade espacial:** se um endereço é acessado, os vizinhos tendem a ser acessados em seguida (instruções sequenciais, elementos consecutivos de vetor).

O que causa (0,5):

- A própria arquitetura de von Neumann: **dados e instruções são armazenados em blocos de memória adjacentes** e o **processamento é sequencial**, com o PC incrementado a cada ciclo de busca. Por isso a próxima instrução quase sempre está ao lado da atual (`7-teoria-1 de 2.pdf` , slide 7).
- A forma de programar: **laços** repetem as mesmas instruções e variáveis; **vetores, matrizes e registros** são percorridos sequencialmente; a **pilha** concentra variáveis locais e chamadas de sub-rotina; a programação modular/estruturada agrupa código relacionado.
- **Consequência:** é a localidade que torna eficientes a **memória cache**, o **pipelining**, a **paginação** (o conjunto de trabalho cabe em poucas páginas) e os **buffers**. Por isso a execução é "rápida" (Q2).

Comentário sobre a resposta do aluno: a definição ("acessar a mesma área de memória com certa frequência") está razoável, mas incompleta: faltam a ideia de "curto intervalo de tempo" e a distinção temporal/espacial. A explicação da causa é confusa ("as áreas de memória para cada item na ED estarão a uma mesma distância entre si") e **não cita** o armazenamento adjacente nem a execução sequencial, que são a resposta do slide. Nota provável: parcial.

Discursiva 2 (1,0)

Enunciado: Quais os tipos de carregadores existentes? (0,5) Cite o instante em que os endereços são relocados (0,25) e o que é carregado em cada um dos tipos de carregador que citar (0,25).

Resposta modelo:

Tipo	Quando os endereços são relocados	O que carrega
Carregador absoluto (IPL no mainframe, ROM BIOS/bootstrap no PC)	**Não há relocação**: o código é absoluto, com endereços fixos e reais definidos na tradução ou programação. O registro informa o endereço de carga (*load address*) e a carga é feita exatamente ali (slide 16).	O **início do sistema operacional** (bootstrap ou carregador do S.O.). Fica em memória não volátil (ROM/flash) e é ativado ao apertar o botão (slide 15).
Carregador de relocação (estática)	**Em tempo de carga**: o carregador soma a constante de relocação k (endereço inicial do bloco dado pelo S.O.) a cada endereço da lista de endereços relocáveis (`7-teoria-1` , slide 14; `7-teoria-2` , slides 38 e 49).	Um **programa executável** (relocável), gerado pelo ligador e contido em disco, carregado num bloco **contíguo** da RAM. Depois faz PC ← entrypoint.
Linking loader / binder (ligador-carregador)	**Em tempo de carga**: liga, reloca e carrega numa única operação, sem gerar arquivo executável (`7-teoria-2` , slide 48).	**Módulos objeto** relocáveis, ligados diretamente na memória.
Carga e ligação dinâmica (DLL/.so)	**Em tempo de execução** (ou de carga): a rotina é carregada e ligada na primeira chamada. O carregador atualiza a tabela *fixup* (`7-teoria-2` , slides 50–52).	**Rotinas de biblioteca dinâmica** (funções da DLL), compartilhadas por vários programas.

Para os 0,5 do primeiro item, os dois tipos centrais dos slides são **absoluto** e **de relocação**. Citar também linking loader e ligação dinâmica enriquece a resposta. Se quiser, mencione ainda a relocação **dinâmica** por hardware (registrador de base/DAT, paginação), feita **durante a execução** (`8-so-red-2` , slides 19–20).

Comentário sobre a resposta do aluno:

- Carregador absoluto: "não ocorre realocação, e carrega o S.O." está correto, mas faltou dizer onde ele fica (ROM) e que carrega o bootstrap/início do S.O.

- Carregador de relocação: "realocação ocorre depois da memória ser alocada para o sistema operacional" é confuso. O certo é "a memória é alocada **pelo** S.O. **para o programa**, e a relocação ocorre **em tempo de carga**". Ele também **não disse o que é carregado** (um programa executável do disco).
- Nota provável: parcial.

Discursiva 3 (1,0)

Enunciado: Apresente um exemplo e explique como é realizado o mapeamento de um endereço lógico em um endereço real em um ambiente paginado.

Resposta modelo:

Mecanismo (`8-so-red-2 de 2.pdf` , pág. 7–8 / slides 20–21):

A figura do slide 21 mostra o endereço lógico dividido em dois campos, **p** (número da página, os bits mais altos) e **d** (deslocamento dentro da página). Um **RegBase** aponta para o início da **tabela de mapeamento de páginas** do processo. Somando RegBase + p obtém-se a entrada da tabela, que contém o número da **página real** (frame) p'. O endereço real é formado por **p' concatenado com d**.

1. A memória é dividida em páginas de **tamanho fixo**, potência de 2 (alguns KB).
2. O endereço lógico é dividido em **(p, d)**. Com páginas de 2^n bytes, d são os n bits menos significativos e p é o restante.
3. O **DAT** (Dynamic Address Translator, ou MMU) usa o RegBase (início da tabela de páginas do processo) + p para acessar a entrada p da tabela.
4. A entrada fornece o **frame** (página real) p'. Se a página não estiver na memória (bit de presença = 0), ocorre **falta de página**: interrupção, programa de canal, processo bloqueado (Q18).
5. **Endereço real = p' concatenado com d.** É concatenação, e não soma, porque o tamanho da página é potência de 2: $p' \cdot 2^n + d$ equivale a justapor os bits, o que é mais barato. Essa é a resposta à pergunta do slide 21. Na segmentação, como os blocos têm tamanho variável, é preciso **somar** (slide 22).

Exemplo: páginas de 4 KB = 2^{12} bytes, então d tem 12 bits (3 dígitos hexa).

Tabela de páginas do processo:

Página lógica (p)	Frame real (p')	Presente?
0	5	sim

Página lógica (p)	Frame real (p')	Presente?
1	9	sim
2	7	sim
3	–	não

- Endereço lógico **0x2ABC** (10940 em decimal): p = 0x2, d = 0xABC (2748).
- A tabela dá p = 2 → p' = 7.
- Endereço real = 7 concatenado com ABC = **0x7ABC**. Conferindo: 7 × 4096 + 2748 = 28672 + 2748 = **31420**.
- Se o endereço fosse 0x3010 (página 3, ausente), haveria **page fault**: o S.O. bloqueia o processo, cria o programa de canal para trazer a página do disco para um frame livre (ou escolhido pelo LRU), atualiza a tabela e devolve o processo à fila de prontos.

Comentário sobre a resposta do aluno: a resposta é genérica ("checando a página respectiva e o deslocamento") e **não apresenta exemplo**, que é pedido explicitamente no enunciado. Faltam a divisão (p, d), o papel da tabela (p → frame), a **concatenação** e o DAT. Nota provável: baixa.

Discursiva 4 (2,0)

Enunciado: Coloque-se na posição de projetista de S.O. Indique qual é a aplicação característica (**batch**, **real time**, **time sharing**, rede ou distribuído), forma de carregamento de programas na memória (contíguos ou não), tipo (monoprogramado, multiprogramado ou multiprocessado), e estrutura (hierárquica, máquina virtual, multiprocessado, distribuído ou orientado a objetos). Apresente e justifique a **política de processos adequada** (0,5) indicando como ela evita **deadlock** (0,5) e **starvation** (0,5), e a **política de gerência de memória real**. (0,5) (2,0)

O aluno deixou em branco.

Esta questão é **aberta**: não há resposta única. O que conta é a **coerência** entre as escolhas e justificar cada uma. Segue uma resposta modelo completa e uma alternativa.

Resposta modelo A: S.O. time sharing

- **Aplicação característica: time sharing**. Muitos usuários interativos, transações curtas e I/O-intensivas; o objetivo é minimizar o **response time**.
- **Carregamento: não contíguo**. O programa é carregado por partes, sob demanda: só a página do entrypoint no início (`8-so-red-1` , slide 14: "tendência atual é alocar por partes, sob

demanda"; `8-so-red-2` , slide 20).

- **Tipo: multiprogramado**. Vários processos na memória, e a CPU é cotizada entre eles enquanto outros esperam E/S (`7-teoria-1` , slide 10).
- **Estrutura: hierárquica** (em camadas). O nível 0 é o kernel, com tratamento de interrupções e escalonamento; cada camada usa os serviços das inferiores, o que dá bom nível de abstração (`8-so-red-2` , slide 25).
- **Política de processos (0,5): round-robin com time slice (preemptivo)**, opcionalmente com **filas de prioridade** (interativos com prioridade maior). A preempção por time slice garante que nenhum processo monopolize a CPU, o que reduz o response time, que é a métrica mais importante em time sharing (`8-so-red-1` , slide 9).
- **Como evita deadlock (0,5):** deadlock é resolvido na **alocação de recursos**, não no escalonamento da CPU (ver Q20). Adota-se **prevenção** negando uma das condições necessárias:
  - **Ordenação global dos recursos:** os processos só podem pedir recursos em ordem crescente de número, o que impede **espera circular**.
  - Alternativa: **preempção de recursos** (o S.O. pode retirar um recurso de quem espera por outro).
  - Pode-se usar o **algoritmo do banqueiro** (só concede o pedido se o estado continuar seguro).
  - *Evito* a técnica de "só entrar na fila de prontos com todos os recursos", porque ela derruba o throughput (Q16).
  - O acesso a recursos compartilhados é feito em **regiões críticas** protegidas por exclusão mútua (`8-so-red-1` , slide 10).
- **Como evita starvation (0,5):** o round-robin é **justo**: cada processo pronto recebe uma fatia de tempo a cada volta da fila, então nenhum espera indefinidamente pela CPU. Se houver prioridades, aplica-se **envelhecimento (aging)**: a prioridade de quem espera muito aumenta até que ele seja atendido. Na alocação de recursos, as filas de espera são FIFO.
- **Gerência de memória real (0,5): swapping** (slide 18): troca de processos entre memória e disco via interrupção de time slice, com relocação dinâmica por **registrador de base/DAT** (slide 19) para que o processo possa voltar em outra posição. É a técnica de memória **real** que casa com time sharing.
  - *Complemento (memória virtual, não substitui a resposta acima):* **paginação** sob demanda com **LRU** (slides 20–21) elimina a fragmentação externa, facilita a proteção e aproveita a localidade de referência. Se for citá-la, deixe claro que é gerência de memória **virtual**: nos slides, paginação, segmentação e paginação-segmentação estão sob o título "Gerência de Memória Virtual" (slides 20–23), e é exatamente por isso que a Q10 é F.
  - *(Correção feita na reconciliação com o guia `00-GUIA-ESTUDO-SB.md` : a versão anterior indicava paginação como gerência de memória real, o que contradiz os slides e a justificativa*

da própria Q10.)

Resposta modelo B (alternativa): S.O. batch

- **Aplicação:** batch (jobs longos, CPU-intensivos, sem interação; objetivo: **throughput** e turnaround).
- **Carregamento:** contíguo. **Tipo:** multiprogramado. **Estrutura:** hierárquica.
- **Processos:** não preemptivo, FIFO ou **SJF** (menor job primeiro, que maximiza o throughput).
- **Deadlock:** como os jobs declaram seus recursos no cartão de controle, pode-se alocar tudo antes da execução (nega posse e espera) ou usar o banqueiro.
- **Starvation:** o SJF pode deixar jobs longos esperando para sempre, então usa-se **aging** (prioridade cresce com o tempo de espera) ou um limite máximo de espera.
- **Memória real: partições** (fixas, com fila por partição, ou variáveis com *first fit* e compactação/relocação contra a fragmentação; `8-so-red-1` , slide 13; `8-so-red-2` , slide 17).

Dicas para esta questão (vale 2,0, a mais pesada da prova):

- Responda **todos** os itens pedidos: aplicação, carregamento, tipo, estrutura, processos, deadlock, starvation, memória. Cada omissão custa pontos.
- Deixe claro que **round-robin evita starvation, não deadlock**. Deadlock exige uma política de recursos (prevenção ou prevenção dinâmica).
- **Prioridade sozinha causa starvation**. Sempre complemente com **aging**.
- Mantenha a coerência: time sharing com time slice e paginação/swapping; batch com SJF/FIFO e partições; real time com prioridade preemptiva e memória sem paginação (determinismo).

Assuntos cobrados nesta prova

Tópico	Questões	Qtde	Conceitos-chave a dominar
Arquitetura de von Neumann (componentes, ciclo, gargalo, localidade)	Q1, Q2, Q4, Q14, D1	5	4 componentes (CPU, memória, E/S, **barramentos**); CPU = ULA, UC, registradores, PC; ciclo busca–decodificação–busca de operandos–execução; PC ← entrypoint, PC incrementado a cada busca; gargalo CPU×memória (cache, pipelining) × memória×E/S (bufferização, DMA, canais); localidade temporal/espacial causada por

Tópico	Questões	Qtde	Conceitos-chave a dominar
			armazenamento adjacente e execução sequencial
Código absoluto, relocação e carregadores (inclui boot)	Q5, Q7, Q8, Q15, D2	5	Carregador absoluto em ROM/flash (não volátil), IPL, ativado pelo botão; código absoluto ainda usado no boot; relocação estática: endereço = offset + k; relocação viabiliza bibliotecas de código objeto (mas bytecode Java não é isso); carregador de relocação: tamanho, bloco contíguo, carga + relocação, PC ← entrypoint; momento da relocação de cada tipo
Montador	Q6, Q9, Q22	3	Entrada Assembly, saída objeto **relocável** + lista de relocáveis + lista de referências externas; estrutura lógica análise (léxica, sintática, semântica) × síntese (otimização de código, não algorítmica + geração); pré-passo (macros, montagem condicional, comentários); passos físicos
Ligador	Q3, Q13, Q21	3	Função: ligar módulos + resolver referências externas; etapas ler/relocar → tabela global → concatenar → resolver; saída **executável** relocável; linkage editor (gera módulo de carga) × linking loader (não gera arquivo) × ligação dinâmica (DLL, fixup)
Gerência de memória (real e virtual) e tipos de S.O.	Q10, Q11, Q12, Q18, D3	5	Tabela do slide 15 (contíguo × não contíguo, mono × multi); batch = CPU-intensivo, partições; time sharing = interativo, swapping/paginação; paginação: sem fragmentação externa, com interna, proteção; segmentação: tamanho variável, soma; mapeamento (p, d) → p' concatenado com d via tabela de páginas e DAT; page fault → programa de canal + bloqueio
Processos: escalonamento, deadlock,	Q16, Q17, Q19, Q20, D4	5	Metas: max throughput, min turnaround e response time; preemptivo × não preemptivo; RR evita **starvation**, não deadlock; alocar todos os

Tópico	Questões	Qtde	Conceitos-chave a dominar
starvation, região crítica			recursos antes evita deadlock mas derruba o throughput; região crítica e exclusão mútua; aging; projeto coerente de S.O. (aplicação, carregamento, tipo, estrutura, processos, memória)

Total: 22 itens objetivos + 4 discursivas = 26 questões. A distribuição é equilibrada entre "softwares básicos" (von Neumann, carregadores, montador, ligador: 16) e "sistemas operacionais" (memória e processos: 10).

Padrões do professor observados nesta prova:

- Itens copiados do slide com **uma palavra trocada**: absoluto ↔ relocável, volátil ↔ não volátil, CPU×memória ↔ CPU×E/S, executável ↔ não executável, interna ↔ externa.
- Itens com primeira parte verdadeira e **conclusão ou exemplo falso** (Q15, Q19).
- Pares de itens com associações trocadas (Q10/Q11: batch × time sharing).
- A discursiva de projeto de S.O. (2,0) é recorrente: tenha uma resposta pronta e coerente decorada.

Apêndice: comparação com a sua versão (`SB_Prova_Sistemas-EU.pdf`)

Sua versão parte da mesma prova, com as suas respostas sobrepostas. Pontos de atenção:

- **Q6 (F/NS):** concordo com **F**, mas a sua justificativa ("pode ser realizada pelo montador") não é o motivo. O motivo é que a síntese otimiza **o código**, não o algoritmo do programador (slide 31).
- **Q10 (F), Q11 (F), Q12 (F), Q13 (V):** concordo. A justificativa da Q12 ("o descrito é segmentação") está ótima.
- **Q14 (F):** considero **V** mais provável. Omitir o incremento do PC não torna o item falso (ver comentário da Q14). Ponto discutível.
- **Q15 (NS):** dá para marcar **F** com segurança razoável (bytecode Java não é código objeto relocável).
- **Q20:** a sua justificativa ("RR evita starvation") é melhor que a do outro aluno.
- **D3:** você escreveu "endereço real = endereço lógico + registrador base". Isso é a relocação **por registrador de base** (slide 19). Na paginação o RegBase aponta para a **tabela de páginas**, e o

endereço real é **frame concatenado com deslocamento**. Faltou também o exemplo pedido.

- **D4:** coerente (time sharing, contíguo, multiprogramado, swapping, time slice), mas:
  - faltou a **estrutura** (ex.: hierárquica);
  - "liberar recursos quando não forem mais necessários" não é uma política de prevenção de deadlock (cite ordenação de recursos/espera circular, banqueiro etc.);
  - "escalonar por prioridade" **não evita** starvation, **pode causá-la**. É preciso **aging** ou round-robin.